

Linux SHELL 编程

!/bin/bash

- shell 编程
- 交互式shell程序
- shell 程序语法
- 图形化界面



Leonardo da Vinci (1452-1519)

shell 脚本结构

\$./*.sh {

- fork (default)
- source
- exec

*.sh

#!/bin/bash

Shebang

echo "hello world! "

Unix 指令

Shell 变量

- 普通变量 / 环境变量
- 变量命名、赋值、引用
- 数组变量, 变量计算

STDIN



0<
|

exit
N



STDOUT

1>
|



STDERR

2>



返回状态

- **SHELL** 返回状态：判断整个脚本的执行是否成功
 - `$ exit [N]` # 以状态N退出shell
 - `N = [0,255]`
 - 若 `N` 被省略，则退出状态为最后一个执行命令的退出状态
 - 0：正常退出
 - 非0：异常退出，可自定义

返回状态值含义

退出码	说明
0	正常退出
1	一般错误
2	Misuse of shell builtins
...	自定义
126	命令不可执行
127	命令未找到
128	返回值无效
128 + n	返回信号值 n为信号编码
130	脚本被中断 130 = 128 + 2, 2为SIGINT
255*	超出返回值范围

执行shell脚本

- 执行shell脚本的三种方式

1. `$ ./***.sh` # 执行当前文件夹内的脚本

2. `$ /.../.../... .../***.sh` # 通过绝对路径定位并执行脚本

3. `$ ***.sh` # 直接执行，需要脚本所在的路径
列于可搜索路径环境变量 `$PATH` 中

`$ echo $PATH`

脚本变量 I/O

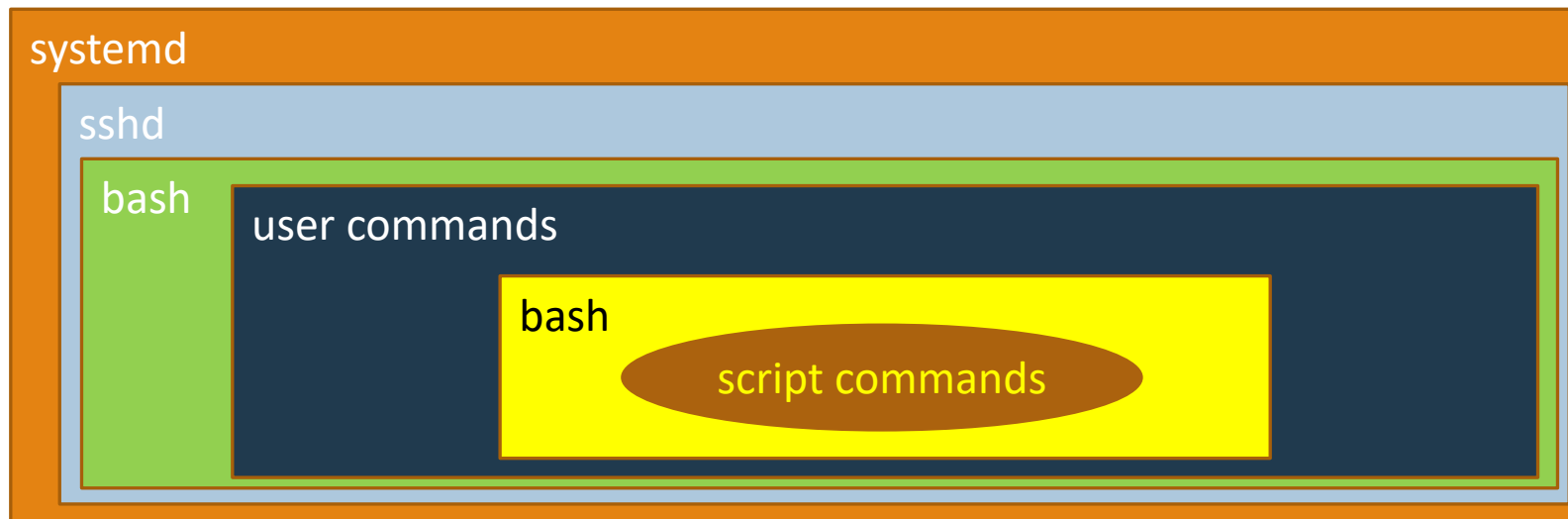
- 输出变量: `echo` 指令
- 输入变量:
 1. `read` 指令: 读取用户输入的数据

```
$ echo "Please input a value for variable 'abc': " ;\  
> read abc ;\  
> echo "the input value is: $abc"
```

- 用户局部变量: 用户在命令行定义的普通变量
- 3. 输入参数: 作为脚本执行命令的一部分
- 4. 命令替换: 读取不同指令的输出被赋值给变量

执行环境与变量继承

- 在Shell中执行脚本，其基本过程是：
 - 在当前shell进程中fork一个shell子进程，用于执行对应程序
 - 程序结束后，再返回当前进程（父进程）。



执行环境与变量继承

- 将脚本路径列入\$PATH时，需要注意pwd的问题，以便调用其它相关文件

```
Bin=`readlink -f "$0"`
```

```
Bin=`dirname "$Bin"`
```

```
Bin=`cd "$Bin"; pwd`
```

#获取脚本的实际路径

```
Source $Bin/env.sh
```


参数变量

- `$0` : 代表脚本程序本身的名字
- `$1, $2 ...` : 代表脚本程序的参数
- `$#` : 代表脚本参数的数量
- `$$` : 代表脚本进程的PID, 常用来标记临时文件
- `$*, @$` : 代表脚本参数的全集
- `$ ./shell.sh`

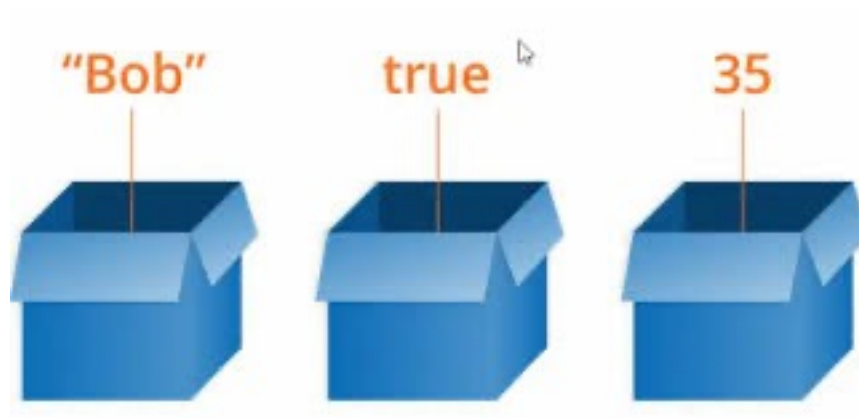
命令替换

- 可以从命令输出中提取信息，并赋值给变量进行进一步处理
 - 反字符对 ``command``: `foo=`date``
 - `$(command)` 方式: `foo=$(date)`

```
#!/bin/bash  
  
# copy the /usr/bin content to a log file  
  
today=$(date +%y%m%d)  
  
ls /usr/bin -al > log.${today}
```

变量引用 & 计算

程序描述了变量进化的过程



变量的引用

- `$ echo ${A}`

Easy ?

变量的引用

Elegant !

- `$ file=/dir1/dir2/dir3/my.file.txt`
- `$ echo ${file#*/}` #拿掉第一條 / 及其左邊的字串: `dir1/dir2/dir3/my.file.txt`
- `$ echo ${file##*/}` #拿掉最後一條 / 及其左邊的字串: `my.file.txt`
- `$ echo ${file#*.}` #拿掉第一個 . 及其左邊的字串: `file.txt`
- `$ echo ${file##*.}` #拿掉最後一個 . 及其左邊的字串: `txt`
- `$ echo ${file%/*}` #拿掉最後條 / 及其右邊的字串: `/dir1/dir2/dir3`
- `$ echo ${file%%/*}` #拿掉第一條 / 及其右邊的字串: (空值)
- `$ echo ${file%.*}` #拿掉最後一個 . 及其右邊的字串: `/dir1/dir2/dir3/my.file`
- `$ echo ${file%%.*}` #拿掉第一個 . 及其右邊的字串: `/dir1/dir2/dir3/my`

变量的引用

: 去掉左边的 (# 在\$ 的左边)
% : 去掉右边的 (% 在\$ 的右边)

- `$ file=/dir1/dir2/dir3/my.file.txt`
- `$ echo ${file#*/}` #拿掉第一條 / 及其左邊的字串: `dir1/dir2/dir3/my.file.txt`
- `$ echo ${file##*/}` #拿掉最後一條 / 及其左邊的字串: `my.file.txt`
- `$ echo ${file#*.}` #拿掉第一個 . 及其左邊的字串: `file.txt`
- `$ echo ${file##*.}` #拿掉最後一個 . 及其左邊的字串: `txt`
- `$ echo ${file%/*}` #拿掉最後條 / 及其右邊的字串: `/dir1/dir2/dir3`
- `$ echo ${file%%/*}` #拿掉第一條 / 及其右邊的字串: (空值)
- `$ echo ${file%.*}` #拿掉最後一個 . 及其右邊的字串: `/dir1/dir2/dir3/my.file`
- `$ echo ${file%%.*}` #拿掉第一個 . 及其右邊的字串: `/dir1/dir2/dir3/my`

变量的引用

- `$ file=/dir1/dir2/dir3/my.file.txt`
- `$ echo ${file:0:5}` # 提取0位置开始的5个字符, /dir1
- `$ echo ${file:5:5}` # 提取5位置开始的5个字符, /dir2
- `$ echo ${#file}` # 计算变量长度

变量的替换

- `$ file=/dir1/dir2/dir3/my.file.txt`
- `$ echo ${file/dir/path}` # 将第一个dir替换为path:
`/path1/dir2/dir3/my.file.txt`
- `$ echo ${file//dir/path}` # 将所有dir替换为path:
`/path1/path2/path3/my.file.txt`

SHELL 数组变量

- `$ A=(a b c def)`
 - 不是 `$ A="a b c def" / A=("a b c def")`
- `$ echo ${A[*]} ; echo ${A[@]} # 全部变量`
- `$ echo ${#A[*]} ; echo ${#A[@]} # 数组长度`
- `$ echo ${A[0]} ; echo ${A[3]} # 特定元素`

变量计算

- `$ A=1 ; B=1; echo $A+$B`
- 结果是 ??

变量计算

- \$ A=1 ; B=1; echo \$A+\$B
 - 结果是 ??

变量计算

- 为实现数学表达式计算，**Bash**提供了：

- `expr` 指令
- 方括号方式
- 双圆括号方式

```
● $ A=2
● $ B=5
● $ expr $A + $B
  7
● $ expr $A * $B
  ?
```

`expr` 指令能够识别常见的数学和字符串表达式，但部分数学符号在 `shell` 中另有含义，在应用时需要加上转义符，较笨拙。

```
● $ expr $A \* $B
```

变量计算

- 方括号方式:

- `$ A=2 ; B=5 ; echo $[A+B] # or $[A+B]`

- 双圆括号方式:

- `$ A=2 ; B=5 ; echo $((A+B)) # or $((A+B))`

操作符	描述
+	加
-	减
*	乘
/	除

操作符	描述
%	取余
++	自增
--	自减

操作符	描述
<	小于
<=	小于等于
>	大于
>=	大于等于

操作符	描述
=	等于
!=	不等
	或
&	与

变量值变化与比较

- 变量值变化

- `$ A=5; echo $(A++); echo $A`

- `$ B=5; echo $(B--); echo $B`

- 变量值比较

- `$ echo $( $A > $B )` # 1 为 true

- `$ echo $( $A < $B )` # 0 为 false

- ~~`$ echo $( A > B )`~~ # 比较对象为ASCII符

变量计算

- **Bash** 只提供整数计算的数学运算符
- 浮点数值的计算需要通过 **bc** 计算器来实现
- **bc** 默认提供CLI交互方式

<code>\$ bc -q</code>	# 以静默方式使用bc
<code>3.44/5</code>	
<code>0</code>	
<code>scale=4</code>	# 设置计算精度 scale
<code>3.44/5</code>	
<code>.6880</code>	
<code>quit</code>	# 退出bc的交互窗口

变量计算

- 在脚本中使用**bc**，必须使用管道方式：

```
variable=$(echo "options; expression" | bc)
```

```
$ echo "scale=4; 3.23*4" | bc
12.92
$ A=1.2
$ B=$(echo "scale=4; $A*3.2" | bc)
$ echo $B
3.84
```


结构化 命令

`if - then - else`

`for while until`

`case`

`test`

`break`

`continue`

不只有关键字，还有指令

流程图



条件判断

- **test 命令**

- `if test -f /bin/bash ; then echo "hello"; fi`

- **[] 命令**

- `if [ -f /bin/bash ]; then echo "hello"; fi`

- 实际的测试命令为 `[ , ]`只是为了配对, 便于理解

- 注意`[]`前后必须有空格, 否则shell会将它理解为参数的一部分, 而并非关键字

条件判断类型

- 字符串比较

表达式	结果
<code>string1 = string2</code>	两字符串相同为真
<code>string1 != string2</code>	两字符串不同为真
<code>-n string</code>	字符串不为空为真
<code>-z string</code>	字符串为null或空串为真

- ```
if [-z $test]; \
then echo "variable test is not defined" ; \
fi
```

# 条件判断类型

- 算术比较

| 表达式                                      | 结果         |
|------------------------------------------|------------|
| <code>expression1 -eq expression2</code> | 两表达式相等     |
| <code>expression1 -ne expression2</code> | 两表达式不等     |
| <code>expression1 -gt expression2</code> | 前者 大于 后者   |
| <code>expression1 -ge expression2</code> | 前者 大于等于 后者 |
| <code>expression1 -lt expression2</code> | 前者 小于 后者   |
| <code>expression1 -le expression2</code> | 前者 小于等于 后者 |
| <code>! expression</code>                | 取反         |

# 条件判断类型

---

- 文件比较

| 表达式                  | 结果        |
|----------------------|-----------|
| <code>-d file</code> | 文件是一个目录   |
| <code>-e file</code> | 文件存在      |
| <code>-f file</code> | 文件是一个普通文件 |
| <code>-r file</code> | 文件可读      |
| <code>-s file</code> | 文件大小不为0   |
| <code>-w file</code> | 文件可写      |
| <code>-x file</code> | 文件可执行     |

- `$ help test` # 可以查看完整的选项

# 条件判断的语句结构

---

```
$ if condition; then
> statements
> elif condition; then
> statements
> else
> statements
> fi
```

# 条件判断的变量引用

---

- `$ read abc # 不赋值而直接回车返回`
- `$ if [ $abc = "hello" ]; then echo true; fi # 因为变量为空, 因此会报错`
- `$ if [ "$abc" = "hello" ]; then echo true; fi`

# 多个条件判断

---

- `if [ $a = 1 ] && [ $b = 2 ] ; then echo true; fi`
- `if [ $a = 1 -a $b = 2 ]; then echo true; fi`



# 多个条件判断

---

- `if [ $a = 1 ] || [ $b = 2 ] ; then echo true; fi`
- `if [ $a = 1 -o $b = 2 ]; then echo true; fi`

# for 循环

---

**for** **variable** **in** **values**

**do**

**statements**

**done**

# for 循环

---

```
#!/bin/bash
```

```
for foo in bar fud 42 ; do
```

```
 echo $foo
```

```
done
```

```
exit 0
```

# for 循环

---

```
#!/bin/bash
```

```
for foo in "bar fud 42" ; do
```

```
 echo $foo
```

```
done
```

```
exit 0
```

# for 循环 使用通配符

---

```
#!/bin/bash
```

```
for file in $(ls *.sh); do
```

```
 wc -l $file
```

```
done
```

```
exit 0
```

# for 循环 使用通配符

---

```
#!/bin/bash
```

```
for file in *.sh; do
```

```
 wc -l $file
```

```
done
```

```
exit 0
```

# while 循环

---

```
while condition do
```

```
 statements
```

```
done
```

# while 循环

---

```
$ cd ~
```

```
$ alibaba.sh
```





# while 循环

---

```
#!/bin/bash
echo -n "Give me a magic word: "
read mgw
while ["$mgw" != "alibaba"]; do
 echo -n "Opps, try again : "
 read mgw
done
echo "Congratulations! "
```



# 参数变量

---

- 构建 `shell` 过滤器 `filter.sh`
  - `while read line; do`
  - `echo "each line is: $line"`
  - `done < "${1:-/dev/stdin}"`
- `./filter.sh file`
- 将输入文件内容作为重定向到`while`循环，逐行输出

# until 循环

---

**until** **condition**

**do**

**statements**

**done**

# break ; continue 命令

---

- **break**: 跳出 `for`, `while`, `until` 循环
- **continue**: 跳到下一个循环

# case 多条件测试

---

```
case variable in
```

```
 pattern [|pattern]...) statements;;
```

```
 pattern [|pattern]...) statements;;
```

```
 ...
```

```
*) statements;;
```

```
esac
```

# Shell 内部命令

---

- `if, for, while, until, case:`  
`shell keywords`
- `test, break, continue:`  
`shell builtin`
- 通过 `help command | keyword` 来查看文档

# shell 函数

# 函数的定义和使用

---

```
function_name () {
 statements
}
```

- `()` 只是用于标记后续`{ }`内的语句被定义为函数体
- 函数必须先被定义，再被引用
- 使用`$1`, `$2`来引用函数参数



# 函数的定义和使用

---

```
function name {
 statements
}
```

- 使用关键字 **function** 来定义函数
- 在函数名 **name** 后，不需要带括号对 ()
- 两种方式都可以，看程序员个人喜好

# 函数的返回值

---

- **echo** 返回函数中的所有STDOUT
- **return** 返回函数执行状态 #不可以用**exit**

```
$ a() { echo "hello"; echo "hello2";
echo "hello3"; return 1; }
```

```
$ b=$(a)
```

```
$ echo $?
```

```
$ echo $b
```

# 配合函数的条件判断

---

1. `if ! a; then`

`echo "fun a returns none zero" ;`

`fi`

- 在条件判断语句中执行函数，并对其返回状态进行判断

2. `if ! a 1>/dev/null ; then`

`echo "true" ;`

`fi`

- 利用重定向，消除无用输出

# 函数参数

---

```
$ c() {
> if [-z $1]; then
> echo "parameter not defined"
> else
> echo $1
> fi
> return 1}
$ b=$(c) ; echo $b
$ b=$(c 10) ; echo $b
```

# 如何获取函数中的输出内容?

---

- 除了 STDOUT, 还有 STDERR

```
$ c() {
> if [-z $1]; then
> echo "parameter not defined" 1>&2
> else
> echo $1
> fi
> return 1}
```

# 如何获取函数中的输出内容?

---

- 使用其他方式进行输出

```
$ db1 () {
> read -p "Enter a value:" value
> echo $[$value * 2]
> }
```

# 在函数中使用变量

---

- 默认情况下，在脚本中定义的任何变量都是全局变量

```
db1 () {
 value=$(($value * 2))
}
```

```
read -p "Enter a value: " value
db1
echo "The new value is: $value"
```

# 在函数中使用变量

- 这会带来一定的危险，需要程序员清楚知道在函数中使用了哪些变量，否则就会扰乱程序的逻辑

```
#!/bin/bash
demonstrating a bad use of variables
```

```
function func1 {
 temp=${ $value + 5 }
 result=${ $temp * 2 }
}
```

```
temp=4
value=6
```

```
func1
echo "The result is $result"
if [$temp -gt $value]
then
 echo "temp is larger"
else
 echo "temp is smaller"
fi
$
$./badtest2
The result is 22
temp is larger
```

**temp** 只是一个临时变量，但由于运行了**func1**，改变了它的值，导致在全局函数中造成了错误的逻辑判断结果。



# 在函数中使用变量

---

- 可以在变量声明前，加上 `local` 关键字，将变量显式地声明为局部变量
- 如果在函数外有相同名字的变量，`shell` 会保证这两个变量的值是分离的

# 向函数传递数组

---

- 默认情况下，将数组变量当作单个参数传递时，函数只会取数组变量的第一个值

```
function testit {
 echo "The parameters are: $@"
 thisarray=$1
 echo "The received array is ${thisarray[*]}"
}
```

```
myarray=(1 2 3 4 5)
echo "The original array is: ${myarray[*]}"
testit $myarray
```

# 向函数传递数组

---

- 需要将数组变量的值分解成单个的值作为参数来进行传递，并在函数内将它们重组为一个数组

```
function testit {
 local newarray
 newarray=(; 'echo "$@"')
 echo "The new array value is: ${newarray[*]}"
}
```

```
myarray=(1 2 3 4 5)
echo "The original array is ${myarray[*]}"
testit ${myarray[*]}
```

- 从函数中传回数组变量也是类似的方法

# GNU shtool 库

---

- **GNU shtool shell**脚本函数库提供了一些简单的**shell**脚本函数，可以用来完成日常的**shell**功能，例如处理临时文件和目录或者格式化输出显示。

<https://ftp.gnu.org/gnu/shtool/>

# GNU shtool 库

表17-1 shtool库函数

| 函 数      | 描 述                               |
|----------|-----------------------------------|
| Arx      | 创建归档文件（包含一些扩展功能）                  |
| Echo     | 显示字符串，并提供了一些扩展构件                  |
| fixperm  | 改变目录树中的文件权限                       |
| install  | 安装脚本或文件                           |
| mdate    | 显示文件或目录的修改时间                      |
| mkdir    | 创建一个或更多目录                         |
| Mkln     | 使用相对路径创建链接                        |
| mkshadow | 创建一棵阴影树                           |
| move     | 带有替换功能的文件移动                       |
| Path     | 处理程序路径                            |
| platform | 显示平台标识                            |
| Prop     | 显示一个带有动画效果的进度条                    |
| rotate   | 转置日志文件                            |
| Scpp     | 共享的C预处理器                          |
| Slo      | 根据库的类别，分离链接器选项                    |
| Subst    | 使用sed的替换操作                        |
| Table    | 以表格的形式显示由字段分隔（field-separated）的数据 |
| tarball  | 从文件和目录中创建tar文件                    |
| version  | 创建版本信息文件                          |

# 文件操作 & 输入输出重 定向

# 重定向符

---

- 文件描述符 (`file descriptor`) : 文件的数字标识
  - `STDIN`: 0
  - `STDOUT`: 1
  - `STDERR`: 2
- `shell` 中最多支持9个文件描述符 0 ~ 8
- 可以通过关联文件描述符与文件, 实现`shell`的文件操作

# 读取文件

---

● `$ ./readfile.sh data`

...

```
exec 0< $file
```

# 关联文件与标准输入

```
while read line; do
```

# read从STDIN读取数据,  
即是从关联文件中按行读取

```
 ...
```

```
done
```



# 输出到文件

---

- `$ ./writeFile.sh log.txt`

- `$ cat log.txt`

...

`exec 3> $file`

# 关联描述符与输出文件

...

`echo "... " >&3`

# 将写入文件描述符3的内容  
重定向到关联文件

# 关闭文件

---

● `$ ./closeFile.sh log.txt`

...

`exec 3> $file`

# 关联描述符与输出文件

...

`exec 3>&-`

# 关闭关联文件

# 创建临时文件

---

- 使用 `mktemp` 可以创建临时文件（默认在 `/tmp` 目录）。该文件不使用默认的 `umask`，而只是设置 `rw` 权限给 `owner` 用户
- 使用时可以设置一个文件名模板，系统会用任意字符码替换 `x` 掩码
- `$ mktemp testing.XXXXXXX`

# 高阶 shell 编程

# Read 超时

---

- 可以通过在使用**read**命令时，设置一个 **-t** 超时选项，用于指定等待输入的秒数，过期后会返回一个非零的退出状态码

```
$ cat test25.sh
#!/bin/bash
timing the data entry
#
if read -t 5 -p "Please enter your name: " name
then
 echo "Hello $name, welcome to my script"
else
 echo
 echo "Sorry, too slow! "
fi
-
```

# 隐藏方式读取

---

- 在某些场合，例如读取用户输入的密码，需要对其数据进行隐藏，可以使用 `-s` 选项
- ```
read -s -p "Enter your password: " pass  
echo "Is your password really $pass ?"
```

捕获信号

- 通过 `trap` 指令可以在`shell` 执行过程中捕获信号

`trap commands signals`

- `$/trap.sh`

移动变量

- `$ ./shift.sh 1 2 3 4 5 6 ...`
- # 如何读取所有未知数量的输入参数?
- `shift [N]`:
 - 模拟出栈操作, 抛出 `N` 位置的输入参数

参数设置与处理

- `getopts` : 用于获取用户参数
- `$ type getopts` # 是一个 `shell builtin`
- `$ getopts "ha:bc:" optKey`
 - 单字母 (h、b) : 普通选项
 - 附带 ":" (a、c) : 期待参数
 - 取参过程中, 参数项取到`optKey`中, 参数值取到`OPTARG`中, 同时`OPTIND` (默认为1) 不断增加
- `$ ./args.sh`

常见的SHELL选项

表14-1 常用的Linux命令选项

选 项	描 述
-a	显示所有对象
-c	生成一个计数
-d	指定一个目录
-e	扩展一个对象
-f	指定读入数据的文件
-h	显示命令的帮助信息
-i	忽略文本大小写
-l	产生输出的长格式版本
-n	使用非交互模式（批处理）
-o	将所有输出重定向到的指定的输出文件
-q	以安静模式运行
-r	递归地处理目录和文件
-s	以安静模式运行
-v	生成详细输出
-x	排除某个对象
-y	对所有问题回答yes

重定向-界定符 (Heredoc)

- **Here Document**: 作为首尾的界定符, 用于在重定向过程中标识多行输入的起止
- ```
$ tr a-z A-Z << EOF
> one two three
> four five
> six seven eight
> EOF
```
- 通常用EOF, 但实际上任意字符均可以, 如 "AAA"

# 重定向-界定符 (Heredoc)

---

- 利用HereDoc 在命令行中输入多行文本并保存到一个文件中去
- ```
$ cat > testFile << EOF  
> hello world!  
> EOF
```
- ```
$ cat > testFile
> hello world!
> ^D
```

# 重定向-界定符 (Heredoc)

---

- 利用HereDoc 一次性输入多个指令并执行

```
●$ sftp root@host131 <<EOF
●> pwd
●> ls
●> get aa
●> exit
●> EOF
●Connected to root@host131.
●sftp> pwd
●Remote working directory: /root
●sftp> ls
●aa anaconda-ks.cfg
●sftp> get aa
●Fetching /root/aa to aa
●/root/aa
100% 9 7.6KB/s 00:00
●sftp> exit
```

# 命令行中的GUI

---

- `dialog` 是运行在Linux控制台上的图形化工具
- `$ dialog --msgbox "Hello World" 9 18`

| 类型  | 选项                         |
|-----|----------------------------|
| 消息框 | <code>--msgbox</code>      |
| 复选框 | <code>--checklist</code>   |
| 信息框 | <code>--infobox</code>     |
| 输入框 | <code>--inputbox</code>    |
| 密码框 | <code>--passwordbox</code> |

| 类型   | 选项                       |
|------|--------------------------|
| 菜单   | <code>--menu</code>      |
| 单选框  | <code>--radiolist</code> |
| 文本框  | <code>--textbox</code>   |
| 是/否框 | <code>--yesno</code>     |
| 时间设置 | <code>--timebox</code>   |

# 命令行中的GUI

---

- `$ dialog --title "Check me" --checklist  
"Pick Numbers" 15 25 3 1 "one" "off" 2  
"two" "on" 3 "three" "off"`

```
--checklist <text> <height> <width> <list height>
<tag1> <item1> <status1>...
```

- `$ dialog --help` #查看不同部件的参数

# 命令行中的GUI

---

- **dialog** 提供两种形式的输出
  - **退出状态码** `$?`: 如果用户选择了**Yes/OK**按钮, 命令返回的退出状态码为0。如果是**Cancel/No**按钮, 返回退出状态码为1。
  - **STDERR**: 用户在菜单选择框的选择结果会被发送到**STDERR**, 可以通过将**STDERR**输出重定向到另一个文件或文件描述符中的方式进行读取。



# 实践平台 中的脚本 示例

`createBatchUser.sh`

`groupon.sh`

`vote.sh`

`statistics.sh`

`i_am_here`

# Advanced Bash-Scripting Guide

---

- 《高级Bash脚本编程指南》
- **Mendel Cooper**
- <http://www.tldp.org/LDP/abs/html/index.html>
- <https://www.gitbook.com/book/imcmy/advanced-bash-scripting-guide-in-chinese/details>

